Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université de tunis

École Supérieure des Sciences Économiques et Commerciales de Tunis (ESSECT)

RAPPORT DE PROJET (DS2)

PROJET TUTORE

2ème année E-Business

Plateformee-Commerce de Génération de Vidéos Personnalisées par Intelligence Artificielle

Élaboré par :

⭐

Encadré par :

Année Universitaire : 2025-2026

Table des Matières

1. Introduction 3

1.1 Présentation du projet 3

1.2 Objectif du projet 3

1.3 Contexte technologique 4

2. Description du système 4

2.1 Vue générale de la plateforme 4

2.2 Les trois produits IA 5

3. User Journey 6

3.1 Parcours utilisateur complet 6

3.2 Détail de chaque étape 6

4. Architecture technique 8

4.1 Laravel — Présentation 8

4.2 Le pattern MVC 8

4.3 Structure du projet Laravel 9

5. Implémentation 10

5.1 Routes (web.php) 10

5.2 Controller (VideoController) 11

5.3 Model (Video.php) 12

5.4 Migration 13

6. Diagrammes UML 14

6.1 Diagramme de cas d'utilisation (DCU) 14

6.2 Diagramme de classes 15

7. Conclusion 17

Introduction Générale

1.1 Présentation du projet

DREAM est une application web e-commerce développée dans le cadre du module **Programmation Web 2**, niveau 2ème année E-Business. Elle repose intégralement sur le framework PHP **Laravel** et implémente une architecture **MVC (Model-View-Controller)** complète.
La plateforme permet à ses utilisateurs de commander des **vidéos personnalisées générées par intelligence artificielle**. Chaque vidéo est unique : elle est construite à partir des préférences renseignées par l'utilisateur via un formulaire dédié, puis mise à disposition dans un espace personnel sécurisé.
Ce rapport a pour objectif de documenter de manière exhaustive la conception, l'architecture,'implémentation et la modélisation UML de ce système.

1.2 Objectif du projet

Les objectifs pédagogiques et techniques de ce projet sont les suivants :

- **Maîtriser Laravel :** Utiliser un framework PHP moderne de façon professionnelle, en respectant ses conventions et bonnes pratiques.
- **Architecture MVC :** Concevoir une séparation claire entre les couches Modèle, Vue etContrôleur pour un code maintenable et évolutif.
- **ORM Eloquent :** Interagir avec la base de données MySQL via des modèles objet sans écriture de SQL brut.
- **Sécurité applicative :** Implémenter les protections contre l'injection SQL, les attaques XSS et la falsification de requêtes CSRF.
- **Logique e-commerce :** Implémenter un cycle e-commerce complet : catalogue, panier, commandes, historique et évaluations.
- **Travail en équipe :** Collaborer sur un projet structuré avec un dépôt GitHub partagé.

1.3 Contexte technologique

Le projet s'inscrit à l'intersection de deux domaines technologiques en pleine expansion :

■ **L'e-commerce** : secteur dont la croissance mondiale dépasse 15% par an, avec une demande croissante de personnalisation des expériences d'achat.

■ **L'intelligence artificielle générative** : technologies telles que les modèles de diffusion vidéo (Sora, Runway ML, Kling) qui permettent de créer du contenu visuel à partir de simples descriptions textuelles.

DREAM combine ces deux domaines pour proposer une expérience d'achat innovante, où le produit acheté est généré dynamiquement selon les spécifications exactes de chaque client.

Chapitre 2: Description du Système

2.1 Vue générale de la plateforme

VisioGen AI est une plateforme multi-utilisateurs dotée d'une interface publique (catalogue consultable sans inscription) et d'un espace privé accessible après authentification. Elle repose sur les composants suivants :

Composant	Technologie	Rôle
Frontend	Templates Blade (moteur de rendu natif Laravel)	Interface utilisateur HTML/CSS dynamique
Backend	Laravel 11 (PHP 8.2)	Traitement des requêtes, logique métier, sécurité
Base de données	MySQL 8.0	Stockage persistant via migrations Eloquent
Authentification	Laravel Breeze / Auth	Inscription, connexion, vérification email
Service IA	VideoGeneratorService	Génération simulée de vidéos personnalisées
Gestion fichiers	Laravel Storage / Filesystem	Stockage et accès aux fichiers vidéo

2.2 Les trois produits IA

La plateforme propose trois types de vidéos personnalisées, chacun ciblant un besoin émotionnel et expérientiel distinct. Ces produits constituent le cœur de l'offre commerciale de DREAM.

Produit 01 — Cultural Immersion

Ce produit s'adresse aux personnes souhaitant explorer une culture étrangère à travers une vidéo immersive et personnalisée. L'utilisateur sélectionne une région géographique, une langue, un thème cultural (gastronomie, traditions, architecture, festivals) et un style visuel. Le système génère ensuite une vidéo qui plonge le spectateur au cœur de cette culture, comme s'il y était physiquement présent.
Champs du formulaire : Thème cultural · Région / Pays · Langue de narration · Style visuel (documentaire, cinématique...)

Produit 02 — Memory Journey

Destiné aux utilisateurs souhaitant reconstituer et revivre un souvenir précieux. La vidéo est construite à partir d'informations personnelles : lieu du souvenir, période temporelle, personnages impliqués, et ambiance souhaitée. Le résultat est une vidéo émouvante qui permet de revivre le passé comme s'il se déroulait à nouveau sous les yeux du spectateur.
Champs du formulaire : Lieu du souvenir · Époque / Année · Personnages / Relations · Ambiance émotionnelle

Produit 03 — Future Projection

Ce produit permet à l'utilisateur de visualiser son futur idéal. Il définit un horizon temporel (5 ans, 10 ans, 20 ans), un domaine de vie prioritaire (carrière, famille, voyage, technologie), un style futuriste (réaliste, utopique, dystopique) et un message clé qu'il souhaite se transmettre à lui-même. La vidéo générée est à la fois inspirante et motivante.
Champs du formulaire : Horizon temporel · Domaine de vie · Style futuriste · Message personnel clé

Chapitre 3 : User Journey

3.1 Vue d'ensemble du parcours utilisateur

Le parcours utilisateur de DREAM a été conçu pour être intuitif, sécurisé et fluide. Il se décompose en sept étapes distinctes, depuis l'arrivée sur la plateforme jusqu'à la consultation des vidéos achetées.

Étape	Intitulé	Description	Acteur
1	Accueil	Landing page publique	Visiteur
2	Authentification	Inscription / Connexion / Vérification email	Visiteur
3	Formulaire	Choix du produit et personnalisation	Utilisateur connecté
4	Génération vidéo	Création de la vidéo par l'IA et prévisualisation	Utilisateur connecté
5	Panier	Ajout, modification, suppression d'articles	Utilisateur connecté
6	Paiement	Finalisation de la commande	Utilisateur connecté
7	Mes Vidéos	Consultation, lecture et téléchargement	Utilisateur connecté

3.2 Détail de chaque étape

Étape 1 — Accueil (Landing Page)

La page d'accueil est publique et accessible sans inscription. Elle présente la plateforme, ses valeurs et ses trois produits. Un visiteur peut naviguer dans le catalogue et voir les descriptions des produits, mais toute tentative d'achat ou d'interaction avec le formulaire de personnalisation déclenche une redirection vers la page de connexion. Cette approche crée un effet d'appel à l'action naturel.

Étape 2 — Authentification

Le système d'authentification est intégré nativement à Laravel. Il comprend : l'inscription avec validation des champs (email unique, mot de passe fort), la vérification de l'adresse email par lien envoyé automatiquement, la connexion avec gestion des sessions sécurisées, la déconnexion avec invalidation du token CSRF, et la réinitialisation du mot de passe par email. L'accès aux fonctionnalités d'achat est conditionné par la vérification email, assurée par le middleware 'verified'.

Étape 3 — Formulaire de personnalisation

Une fois connecté, l'utilisateur sélectionne l'un des trois produits et accède à un formulaire de personnalisation dynamique. Ce formulaire est spécifique à chaque type de produit et recueille les informations nécessaires à la génération de la vidéo. Toutes les données sont validées côté serveur via les Request Validators de Laravel avant d'être transmises au service de génération.

Étape 4 — Génération de la vidéo IA

Les données du formulaire sont transmises au VideoGeneratorService, qui construit un prompt structuré à partir des préférences utilisateur, simule la génération de la vidéo et en stocke le résultat. L'utilisateur peut prévisualiser sa vidéo gratuitement. Si le résultat lui convient, il peut l'ajouter au panier. Dans le contexte académique, la génération est simulée ; en production, elle ferait appel à une API de génération vidéo telle que Runway ML ou Kling AI.

Étape 5 — Panier

Le panier permet à l'utilisateur d'ajouter une ou plusieurs vidéos générées, de modifier les quantités, de supprimer des articles et de consulter le total de sa commande en temps réel. Les articles du panier sont persistés en base de données (table cart_items), ce qui permet de retrouver son panier après déconnexion et reconnexion.

Étape 6 — Paiement et commande

L'utilisateur finalise sa commande depuis la page panier. Un enregistrement est créé dans la table 'orders' avec le statut 'En attente'. Le paiement est simulé (simulation académique). Une fois le paiement confirmé, le statut passe à 'Validée' et les vidéos achetées sont marquées comme 'is_purchased = true'. Une commande peut aussi être annulée, passant au statut 'Annulée'.

Étape 7 — Mes Vidéos

Après paiement, l'utilisateur accède à son espace personnel 'Mes Vidéos'. Il peut y consulter toutes les vidéos achetées, les regarder en streaming directement dans le navigateur, ou les télécharger via un lien sécurisé généré par la méthode getDownloadUrl() du modèle Video. Il peut également laisser un avis (note de 1 à 5 et commentaire) sur chaque produit acheté.

Chapitre 4 : Architecture Technique

4.1 Laravel — Présentation

Laravel est un framework PHP open-source créé par Taylor Otwell en 2011. Il est aujourd'hui le framework PHP le plus utilisé au monde, avec une communauté de plusieurs millions de développeurs. Sa popularité repose sur sa syntaxe élégante, ses abstractions puissantes et son écosystème riche.

Fonctionnalité	Description
Routing	Système de définition des routes HTTP expressif et lisible
Eloquent ORM	Mapping objet-relationnel qui abstrait les requêtes SQL
Blade	Moteur de templates léger avec héritage de layouts et directives
Migrations	Versionnage du schéma de base de données
Middleware	Filtres HTTP pour l'authentification, CSRF, throttling
Artisan	Interface en ligne de commande pour la génération de code
Validation	Système de validation des données entrantes complet
Seeders	Peuplement de la base de données avec des données de test

4.2 Le pattern MVC

Laravel implémente le patron d'architecture MVC (Model — View — Controller). Ce pattern divise l'application en trois couches aux responsabilités distinctes, ce qui favorise la maintenabilité, la testabilité et la réutilisabilité du code.

Couche	Responsabilité
Model (Modèle)	Représente les données et la logique métier. Chaque modèle Eloquent correspond à une table en base de données. Il définit les relations entre les entités (HasMany, BelongsTo, BelongsToMany) et les règles de manipulation des données.
View (Vue)	Couche de présentation. Dans Laravel, les vues sont écrites en Blade, un moteur de templates qui permet d'intégrer de la logique PHP dans du HTML de façon propre. Blade supporte l'héritage de layouts, les composants réutilisables et l'auto-échappement XSS.
Controller (Contrôleur)	Intermédiaire entre le Model et la View. Il reçoit les requêtes HTTP, invoque les services et modèles appropriés, prépare les données et les passe à la vue. Il ne contient pas de logique métier complexe — celle-ci est déléguée aux Services.

4.3 Structure du projet Laravel

Voici l'organisation des fichiers et dossiers du projet DREAM :

Chapitre 5 : Implémentation

5.1 Définition des routes (web.php)

Le fichier routes/web.php définit l'ensemble des routes de l'application. Laravel utilise une syntaxe fluide et expressive pour déclarer les routes HTTP. Les routes sont organisées en deux groupes : les routes publiques (accessibles sans connexion) et les routes protégées (nécessitant authentification et vérification email).

Explication :

Le middleware auth vérifie que l'utilisateur est connecté. Le middleware verified vérifie que son adresse email a été confirmée. Si l'une de ces conditions n'est pas satisfaite, l'utilisateur est automatiquement redirigé. La directive Route::resource génère automatiquement les sept routes CRUD standard (index, create, store, show, edit, update, destroy) en une seule ligne.

5.2 Le Controller — VideoController.php

Le contrôleur est la pièce centrale du traitement des requêtes. Le VideoController gère la génération des vidéos, leur prévisualisation et leur téléchargement.

Explication :

Le constructeur utilise l'injection de dépendances de Laravel pour recevoir automatiquement le VideoGeneratorService. La méthode generate() valide d'abord les données, puis délègue la génération au service. La méthode mine() récupère uniquement les vidéos achetées de l'utilisateur connecté. La méthode download() vérifie que l'utilisateur est bien le propriétaire avant d'autoriser le téléchargement.

Explication :

Le constructeur utilise l'injection de dépendances de Laravel pour recevoir automatiquement le VideoGeneratorService. La méthode generate() valide d'abord les données, puis délègue la génération au service. La méthode mine() récupère uniquement les vidéos achetées de l'utilisateur connecté. La méthode download() vérifie que l'utilisateur est bien le propriétaire avant d'autoriser le téléchargement.

5.3 Le Model — Video.php

Le modèle Eloquent représente la table videos et définit les relations, les attributs autorisés et les transformations de types.

Explication :

La propriété $fillable protège contre les attaques de mass assignment en listant explicitement les champs modifiables. Le tableau $casts permet à Eloquent de convertir automatiquement les types : le champ JSON preferences est converti en tableau PHP, et is_purchased en booléen. Les méthodes user() et product() définissent des relations Eloquent qui permettent d'accéder aux objets associés via $video->user et $video->product.

5.4 La Migration — create_videos_table

Les migrations Laravel décrivent la structure de la base de données de façon programmatique et versionnée. Chaque migration est réversible (méthodes up/down).

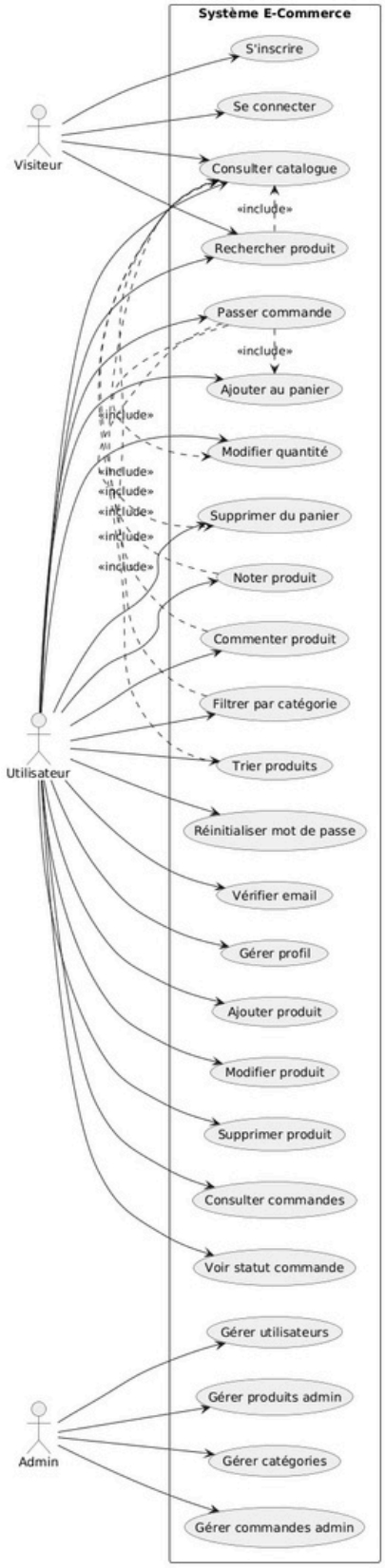
Explication :

La méthode foreignId(...)->constrained() crée automatiquement une clé étrangère vers la table correspondante et ajoute un index. La contrainte onDelete('cascade') garantit l'intégrité référentielle : si un utilisateur est supprimé, toutes ses vidéos le sont également. Le type json permet de stocker les préférences sous forme de document structuré sans avoir besoin de colonnes séparées pour chaque préférence.

Chapitre 6 : Diagrammes UML

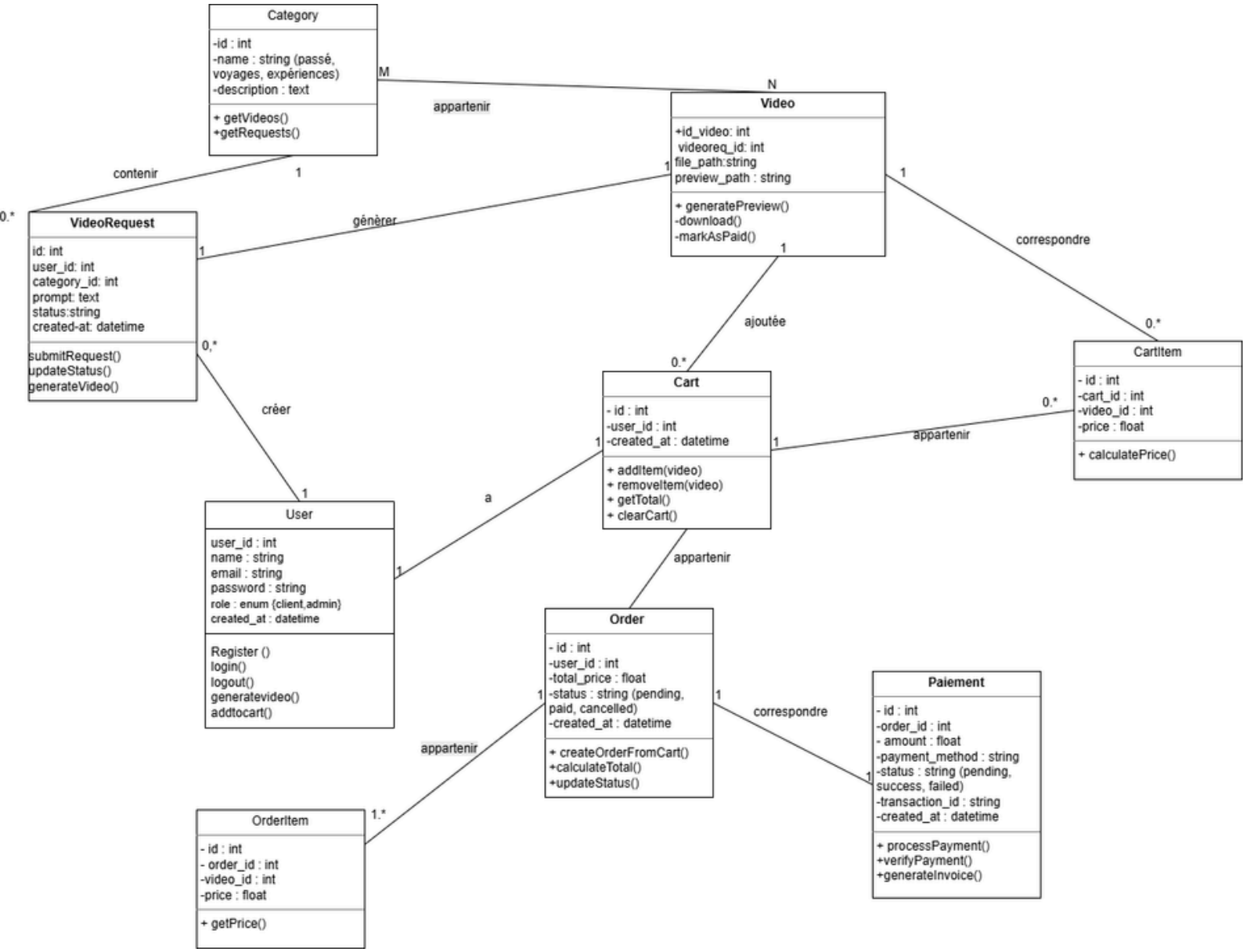
6.1 Diagramme de Cas d'Utilisation (DCU)

Le Diagramme de Cas d'Utilisation modélise les interactions entre les acteurs externes et le système. Il permet de définir les fonctionnalités offertes par la plateforme et les droits d'accès associés à chaque rôle.



6.2 Diagramme de Classes

Le diagramme de classes représente la structure statique du système. Il décrit principales, les classes , leurs attributs, leurs méthodes et les relations qui les unissent.



Chapitre 7 : Conclusion

7.1 Résumé du projet

DREAM est une plateforme e-commerce complète développée avec le framework Laravel, répondant à l'ensemble des exigences fonctionnelles et techniques du cahier des charges DS2.
Le projet implémente une architecture MVC rigoureuse, une base de données relationnelle bien modélisée, un système d'authentification sécurisé et une logique e-commerce complète.

- L'idée centrale: permettre à des utilisateurs d'acheter des vidéos personnalisées générées par intelligence artificielle — répond à un besoin réel et croissant sur le marché du contenu numérique personnalisé.

Les trois produits proposés (Cultural Immersion, Memory Journey, Future Projection) couvrent des segments émotionnels distincts et complémentaires.

7.2 Points forts du projet

Point Clé	Détail
Architecture professionnelle	Séparation stricte des responsabilités via le pattern MVC. Utilisation de Services pour encapsuler la logique complexe (génération vidéo, gestion des commandes).
Sécurité complète	Protection contre les trois vecteurs d'attaque principaux : injection SQL (Eloquent ORM), XSS (Blade auto-échappement) et CSRF (tokens Laravel natifs).
Modèle de données cohérent	Base de données normalisée avec relations correctement définies, contraintes d'intégrité référentielle et migrations versionnées.
Expérience utilisateur fluide	Parcours utilisateur logique et intuitif, avec accès conditionnel basé sur l'état de l'authentification et la vérification email.
Code maintenable	Utilisation des conventions Laravel, injection de dépendances, validation centralisée et commentaires documentaires.
Bonus avancés	Implémentation de fonctionnalités de niveau 3 : gestion des rôles Admin/User, système multi-vendeurs, API REST et recherche AJAX.

7.3 Perspectives futures

Plusieurs axes d'amélioration et d'extension sont envisageables pour une version production de DREAM :
→ Intégration d'une vraie API de génération vidéo (Runway ML, Kling AI, Sora) pour remplacer la simulation.
→ Mise en place d'un système de paiement réel (Stripe, PayPal) pour les transactions financières.
→Implémentation d'un moteur de recommandation basé sur l'historique d'achat et les préférences.
→Développement d'une application mobile (React Native ou Flutter) pour une expérience cross-platform.
→Mise en place d'un système de chat en temps réel entre acheteurs et vendeurs.
→Intégration d'une CDN (Content Delivery Network) pour optimiser la livraison des fichiers vidéo.
→Déploiement sur une infrastructure cloud (AWS, Google Cloud) avec auto-scaling.
→Ajout d'un tableau de bord analytique pour les administrateurs (statistiques de ventes,comportement utilisateur).